

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
```

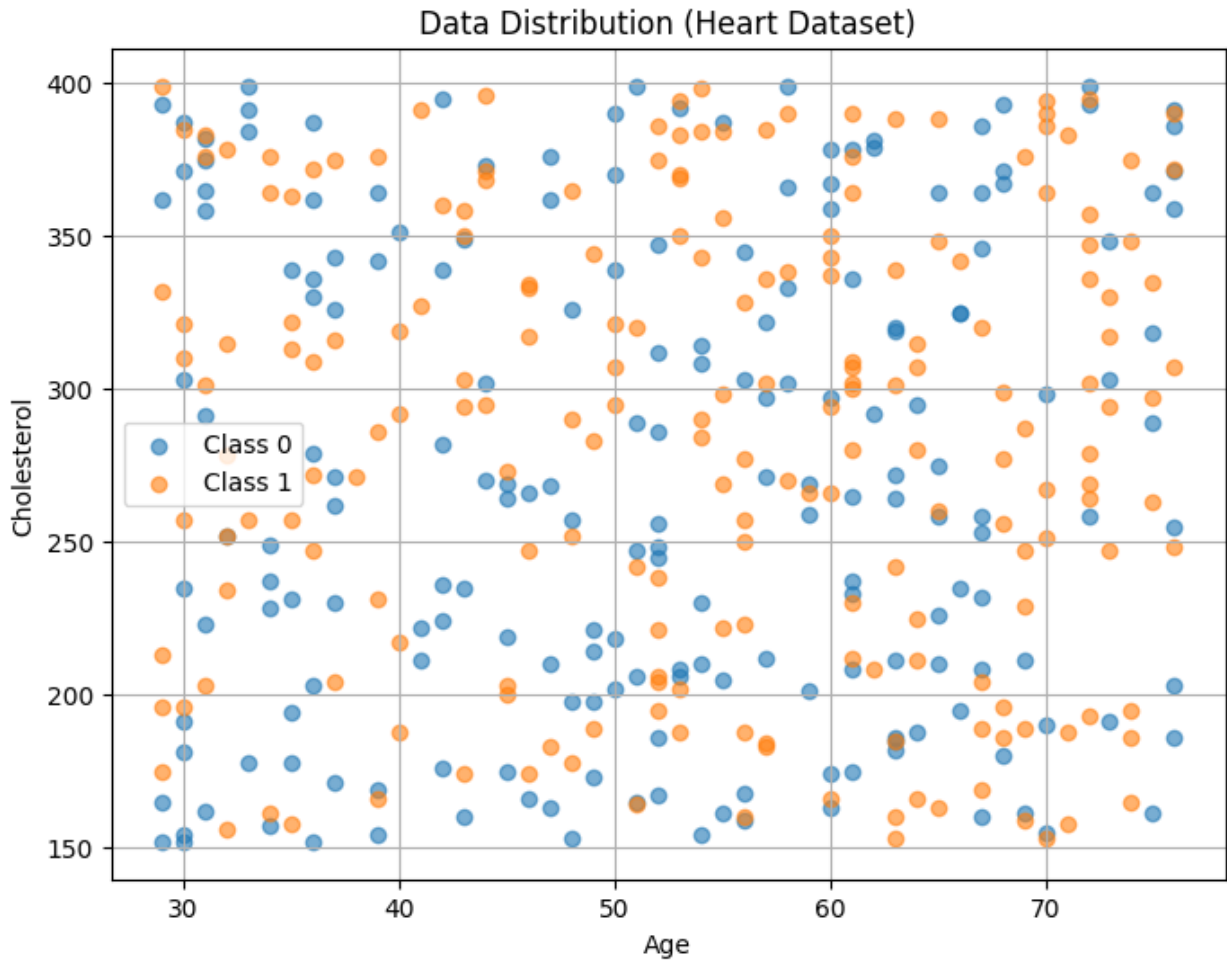
```
df = pd.read_excel("heart_400_rows.xlsx")
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak
0	67	1	3	181	346	0	2	106	0	4.0
1	57	1	0	164	297	1	1	80	0	3.8
2	43	1	1	93	160	0	1	207	1	2.2
3	71	1	2	168	383	1	1	147	0	5.2
4	36	1	0	95	279	0	1	146	1	2.9

	ca	thal	target
0	1	1	0
1	3	1	0
2	0	1	0
3	2	2	1
4	1	3	0

```
X = df[["age", "chol"]].values
y = df["target"].values
```

```
plt.figure(figsize=(8,6))
plt.scatter(X[y==0][:,0], X[y==0][:,1], label="Class 0", alpha=0.6)
plt.scatter(X[y==1][:,0], X[y==1][:,1], label="Class 1", alpha=0.6)
plt.title("Data Distribution (Heart Dataset)")
plt.xlabel("Age")
plt.ylabel("Cholesterol")
plt.legend()
plt.grid()
plt.show()
```



```
def mle_estimate(X, y, c):
    Xc = X[y==c]
    mean = np.mean(Xc, axis=0)
    cov = np.cov(Xc.T)
    return mean, cov

mean0, cov0 = mle_estimate(X, y, 0)
mean1, cov1 = mle_estimate(X, y, 1)

print("Mean (Class 0):", mean0)
print("Mean (Class 1):", mean1)

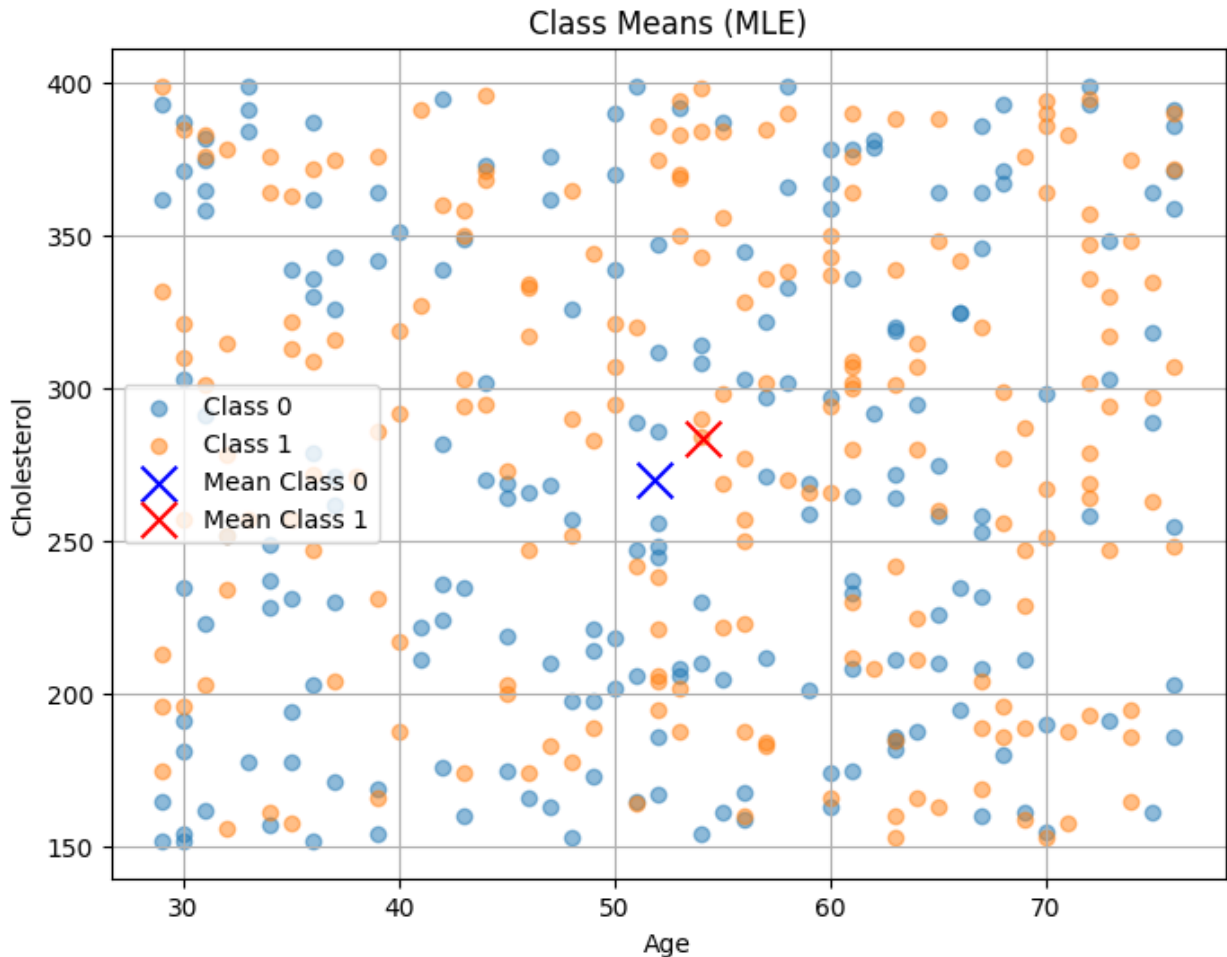
Mean (Class 0): [ 51.86734694 270.00510204]
Mean (Class 1): [ 54.09803922 283.59803922]

plt.figure(figsize=(8,6))
plt.scatter(X[y==0][:,0], X[y==0][:,1], label="Class 0", alpha=0.5)
plt.scatter(X[y==1][:,0], X[y==1][:,1], label="Class 1", alpha=0.5)

plt.scatter(mean0[0], mean0[1], marker="x", s=200, label="Mean Class
0", color="blue")
```

```
plt.scatter(mean1[0], mean1[1], marker="x", s=200, label="Mean Class 1", color="red")

plt.title("Class Means (MLE)")
plt.xlabel("Age")
plt.ylabel("Cholesterol")
plt.legend()
plt.grid()
plt.show()
```



```
def plot_cov_ellipse(mean, cov, ax, color):
    eigenvalues, eigenvectors = np.linalg.eig(cov)
    angle = np.degrees(np.arctan2(*eigenvectors[:, 0][::-1]))

    width, height = 2 * np.sqrt(eigenvalues)

    ellipse = Ellipse(
        xy=mean,
        width=width,
```

```

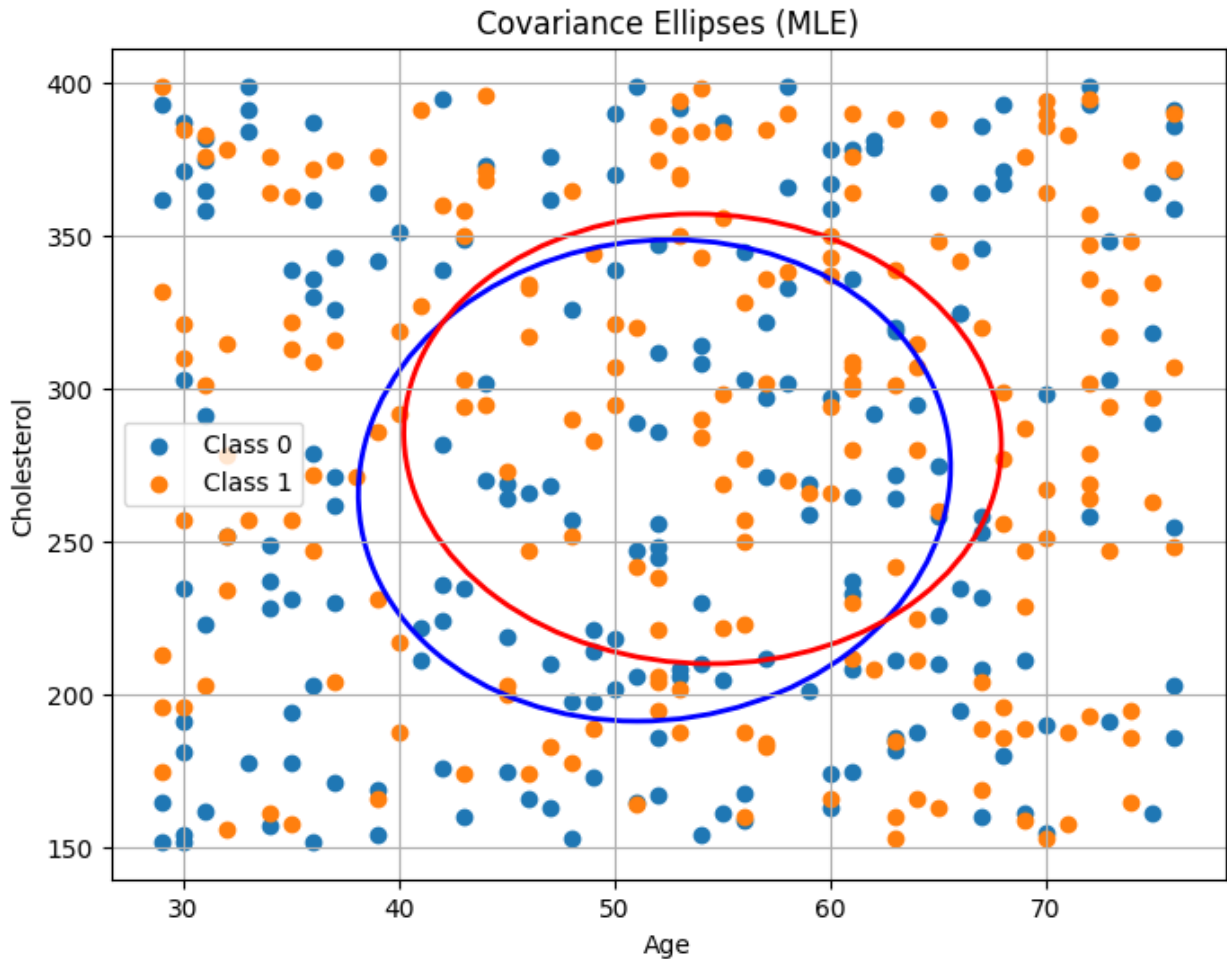
        height=height,
        angle=angle,    # angle must be keyword argument
        edgecolor=color,
        facecolor='none',
        lw=2
    )
    ax.add_patch(ellipse)

fig, ax = plt.subplots(figsize=(8,6))
ax.scatter(X[y==0][:,0], X[y==0][:,1], label="Class 0")
ax.scatter(X[y==1][:,0], X[y==1][:,1], label="Class 1")

plot_cov_ellipse(mean0, cov0, ax, "blue")
plot_cov_ellipse(mean1, cov1, ax, "red")

ax.set_title("Covariance Ellipses (MLE)")
ax.set_xlabel("Age")
ax.set_ylabel("Cholesterol")
ax.legend()
ax.grid()
plt.show()

```



```
def gaussian_likelihood(x, mean, cov):
    d = len(mean)
    det = np.linalg.det(cov)
    inv = np.linalg.inv(cov)
    norm_const = 1 / np.sqrt((2*np.pi)**d * det)
    diff = x - mean
    return norm_const * np.exp(-0.5 * diff.T @ inv @ diff)

def predict_mle(X):
    preds = []
    for x in X:
        L0 = gaussian_likelihood(x, mean0, cov0)
        L1 = gaussian_likelihood(x, mean1, cov1)
        preds.append(0 if L0 > L1 else 1)
    return np.array(preds)

y_pred = predict_mle(X)
print("MLE Classifier Accuracy:", accuracy_score(y, y_pred))

MLE Classifier Accuracy: 0.5575
```

```

xx, yy = np.meshgrid(
    np.linspace(X[:,0].min(), X[:,0].max(), 300),
    np.linspace(X[:,1].min(), X[:,1].max(), 300)
)

grid = np.c_[xx.ravel(), yy.ravel()]
grid_pred = predict_mle(grid).reshape(xx.shape)

plt.figure(figsize=(8,6))
plt.contourf(xx, yy, grid_pred, alpha=0.3, cmap="bwr")

plt.scatter(X[y==0][:,0], X[y==0][:,1], label="Class 0")
plt.scatter(X[y==1][:,0], X[y==1][:,1], label="Class 1")

plt.title("Decision Boundary (MLE Classifier)")
plt.xlabel("Age")
plt.ylabel("Cholesterol")
plt.legend()
plt.grid()
plt.show()

```

